TRAFFIC SIGNAL DETECTION ROBOT

MARS MINI PROJECT

NAME : SHARVANI (PES1UG23AM165)

NAME : NAVYASHREE(PES1UG23AM180)

NAME : NIDHI SHETTY (PES1UG23AM185)

NAME : REVATHI(PES1UG23AM233)

1. Project Overview

This project implements an autonomous traffic signal detection and response robot using ROS2 Jazzy and Gazebo Harmonic. The robot uses a live webcam feed to detect traffic signs in real time, classifies them using a hybrid YOLOv8 + OpenCV detection pipeline, and responds by publishing velocity commands to a simulated differential-drive robot in Gazebo.

1.1 Objectives

- Detect traffic signs from a live camera feed without manual training on the host machine.
- Map detected signs to robot movement commands (GO, STOP, LEFT, RIGHT, SLOW, WAIT).
- Simulate a differential-drive robot responding to those commands inside Gazebo Harmonic.
- Integrate all components into a single ROS2 launch file for one-command startup.

1.2 Technology Stack

Component	Technology	Version / Notes
Robot OS	ROS2 Jazzy	Ubuntu 24.04 (VirtualBox)
Simulator	Gazebo Harmonic	gz sim 8.11.0
ML Detection	YOLOv8n / YOLOv8s (Ultralytics)	COCO pretrained + custom best.pt
Fallback Detection	OpenCV 4.x	HSV colour + shape contour
Dataset	Roboflow traffic-sign-umak0 v8	1,165 images, 56 classes
Training (GPU)	Google Colab T4 GPU	YOLOv8n, 30 epochs, ~20 min
Build Tool	colcon	ament_python package

2. System Architecture

The system consists of three cooperating components connected via ROS2 topics:

Component	Node Name	Subscribes	Publishes
Sign Detector	sign_detector	Webcam (cv2.VideoCapture)	/traffic_sign (String)
Robot Controller	robot_controller	/traffic_sign (String)	/cmd_vel (Twist)
ROS–GZ Bridge	parameter_bridge	/cmd_vel (ROS2 Twist)	gz.msgs.Twist → Gazebo

2.1 Data Flow

Webcam Frame → Sign Detector Node → /traffic_sign topic → Robot Controller → /cmd_vel → ROS-GZ Bridge → Gazebo DiffDrive Plugin → Robot Motion

2.2 Workspace Structure

```
traffic_robot_ws/
  src/traffic_robot/
    traffic_robot/            # Python nodes
      sign_detector.py        # Camera + YOLO + OpenCV detection
      robot_controller.py     # Command → Twist velocity mapping
    worlds/traffic_world.sdf # Gazebo environment
    urdf/robot.sdf            # Differential-drive robot model
    launch/traffic_robot.launch.py
    setup.py / package.xml
```

3. Traffic Sign Detection

3.1 Detection Strategy

A hybrid two-layer approach is used to maximise reliability:

- Layer 1 — YOLOv8 (primary): A custom-trained best.pt model identifies specific traffic sign classes by shape and pattern, independent of colour alone.
- Layer 2 — OpenCV (fallback): HSV colour segmentation combined with contour-based shape classification detects signs when YOLO confidence is low.
- Merger logic: YOLO detections take priority. OpenCV detections are only added when they do not overlap with any YOLO bounding box.

3.2 YOLO Model Details

Architecture: YOLOv8n (nano), trained on Google Colab using a free T4 GPU (~20 minutes for 30 epochs).

Dataset — Roboflow traffic-sign-umak0 v8 (56 classes, 1,165 images):

- Stop_Sign, No-Entry
- Left-Only, Keep-Left, no left turn, Dangerous-Left-Curve-Ahead
- Right-Only, Keep-Right, no right turn, Dangerous-Right-Curve-Ahead
- 20/40/60/80/110-speed-limit, 60/80/110-Speed-Camera
- traffic lights ahead, No-Parking-Or-Standing, No-U-turn
- Give-Way, one way, Pass either side (and 35+ more classes)

3.3 OpenCV Shape Classification

The OpenCV detector segments colour regions in HSV space, then classifies contours by vertex count using approxPolyDP:

Colour	Shape	Command Issued
Green	Circle	GO
Red	Octagon	STOP

Red	Circle	STOP
Yellow	Triangle	WAIT
Yellow	Circle	SLOW
Blue	Rectangle	GO (directional)

3.4 Command-to-Sign Mapping

Command	Source	Triggered By (YOLO class names)
STOP	YOLO	Stop_Sign, No-Entry, stop sign
GO	YOLO	Give-Way, one way, Pass either side, traffic light
LEFT	YOLO	Left-Only, Keep-Left, no left turn, Dangerous-Left-Curve-Ahead
RIGHT	YOLO	Right-Only, Keep-Right, no right turn, Dangerous-Right-Curve-Ahead
SLOW	YOLO	20/40/60/80/110-speed-limit
WAIT	YOLO	traffic lights ahead, No-Parking-Or-Standing, No-U-turn
GO	OpenCV	Green circle
STOP	OpenCV	Red octagon, Red circle
WAIT	OpenCV	Yellow triangle
SLOW	OpenCV	Yellow circle

4. Robot Controller

4.1 Velocity Mapping

The robot_controller node subscribes to /traffic_sign and publishes Twist messages to /cmd_vel at 10 Hz:

Command	linear.x (m/s)	angular.z (rad/s)	Behaviour
GO	0.5	0.0	Forward
STOP	0.0	0.0	Stationary
LEFT	0.3	+0.8	Curve left
RIGHT	0.3	-0.8	Curve right
SLOW	0.15	0.0	Slow forward
WAIT	0.0	0.0	Stop 5 s then GO

4.2 WAIT Behaviour

When the WAIT command is received, the controller spawns a background thread that holds the robot stationary for 5 seconds before automatically issuing a GO command and clearing the waiting flag. This prevents the robot from responding to new signs during the wait period.

4.3 Robot Model (SDF)

The robot is a differential-drive robot defined in robot.sdf:

- Body: 0.4 m × 0.3 m × 0.15 m box, mass 2 kg.
- Drive wheels: Two cylindrical wheels (radius 0.08 m, separation 0.36 m) with revolute joints and friction μ = 1.0.
- Caster: Rear spherical caster (radius 0.04 m) with near-zero friction for free rotation.
- Plugin: gz-sim-diff-drive-system subscribes to cmd_vel and drives left_wheel_joint and right_wheel_joint.

5. Gazebo World (traffic_world.sdf)

The simulated environment represents an urban road scene:

- Main road: 60 m × 7 m dark-asphalt box.
- Centre-line dashes: Five white dashes at −20, −10, 0, +10, +20 m.
- Pavements: Two 60 m × 2 m beige sidewalks flanking the road.
- Buildings: Five coloured box structures of varying heights (2–5 m) placed on both sides of the road.
- Lighting: Single directional sun light with shadows enabled.
- Physics: Ignition Physics plugin, 1 ms step size, real-time factor 1.0.
- Robot spawn point: x = −20 m (western end of road), heading east.

6. Launch File & Build

6.1 Launch Sequence (traffic_robot.launch.py)

The launch file orchestrates four components with staggered timers:

Delay	Action	Detail
0 s	Launch Gazebo	gz sim -r traffic_world.sdf
5 s	Spawn Robot	gz service /world/traffic_world/create with robot.sdf
8 s	Start Bridge	ros_gz_bridge: /cmd_vel Twist ↔ gz.msgs.Twist
10 s	Start Controller	robot_controller node

6.2 Running the Project

Terminal 1 — Launch Gazebo + robot + controller:

```
source /opt/ros/jazzy/setup.bash
source ~/traffic_robot_ws/install/setup.bash
ros2 launch traffic_robot traffic_robot.launch.py
```

Terminal 2 — Start the sign detector (webcam):

```
source /opt/ros/jazzy/setup.bash
source ~/traffic_robot_ws/install/setup.bash
ros2 run traffic_robot sign_detector
```

Terminal 3 — Monitor commands (optional):

```
ros2 topic echo /traffic_sign
```

6.3 Build Commands

```
cd ~/traffic_robot_ws
colcon build --packages-select traffic_robot
source install/setup.bash
```

7. Challenges & Solutions

Challenge	Root Cause	Solution
Training took 4–12 hours on CPU	18,640-image dataset too large for local CPU	Switched to Google Colab T4 GPU; reduced to 1,165-image v8 dataset; finished in ~20 min
Dataset missing valid/ and test/ folders	Roboflow v7/v8 download had 0-split validation	Programmatically created folders and copied 100 training images as validation split
Roboflow API 403 errors	Model belonged to a different workspace; public WiFi firewall	Switched to mobile hotspot; adopted locally-trained best.pt to eliminate API dependency
best.pt corrupted via WhatsApp transfer	WhatsApp unzips .pt files into raw PyTorch data directories	Transferred via Google Drive / direct download; removed the incorrectly-extracted best/ directory
OpenCV GUI error (cvShowImage)	opencv-python-headless installed instead of full opencv-python	Reinstalled opencv-python with GTK support; installed libgtk2.0-dev and libgtk-3-dev
NumPy 2.x compatibility error	System NumPy 2.3 incompatible with pre-compiled OpenCV extension	Pinned numpy<2 via pip3 install
Pure colour detection unreliable (green trees etc.)	HSV thresholds trigger on natural scene colours	Combined YOLO (primary) with shape-constrained OpenCV (fallback); YOLO takes priority on overlapping regions

8. Project Status & Remaining Steps

8.1 Completed

- ROS2 Jazzy workspace created and built successfully.
- Gazebo Harmonic world with road, buildings, and spawn point created.
- Differential-drive robot SDF with DiffDrive plugin created.
- sign_detector.py — hybrid YOLO + OpenCV detection node implemented.
- robot_controller.py — command-to-velocity mapping node with WAIT timer implemented.
- Launch file with staggered startup sequence implemented.
- YOLOv8n model trained on Colab GPU (30 epochs, 1,165 images, 56 classes).
- Package built with colcon; executables installed as sign_detector and robot_controller.

8.2 Pending

- Place trained best.pt at ~/best.pt so the sign_detector loads the custom model (currently falls back to base YOLOv8n which only knows stop sign and traffic light from COCO).
- End-to-end test: launch Gazebo, run sign_detector, show signs to webcam, verify robot responds.
- Optional: add traffic sign 3D models to the Gazebo world so the robot detects signs autonomously while driving.
- Optional: add camera sensor to robot.sdf for fully autonomous vision without a physical webcam.

9. Signs to Show to the Webcam

When running the sign detector, hold printed or phone-displayed traffic sign images approximately 30–50 cm from the webcam with good lighting. The sign should fill at least 30% of the frame. Hold it steady for 2–3 seconds.

Sign to Show	Robot Command	Robot Behaviour
Stop Sign (red octagon)	STOP	Robot halts immediately
Give Way / One Way	GO	Robot moves forward at 0.5 m/s
Left-Only / Keep-Left arrow	LEFT	Robot curves left at 0.3 m/s + 0.8 rad/s
Right-Only / Keep-Right arrow	RIGHT	Robot curves right at 0.3 m/s − 0.8 rad/s
Speed Limit (20/40/60/80)	SLOW	Robot moves slowly at 0.15 m/s
Traffic Light / No Parking	WAIT	Robot stops for 5 seconds then resumes GO

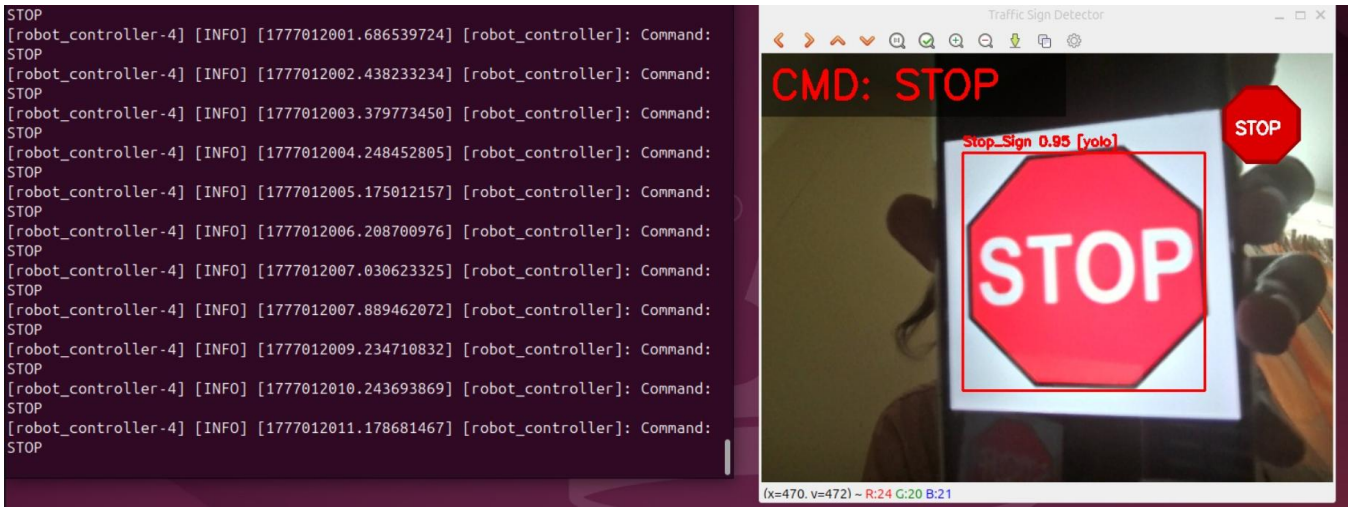
Google Image Search terms for phone display:

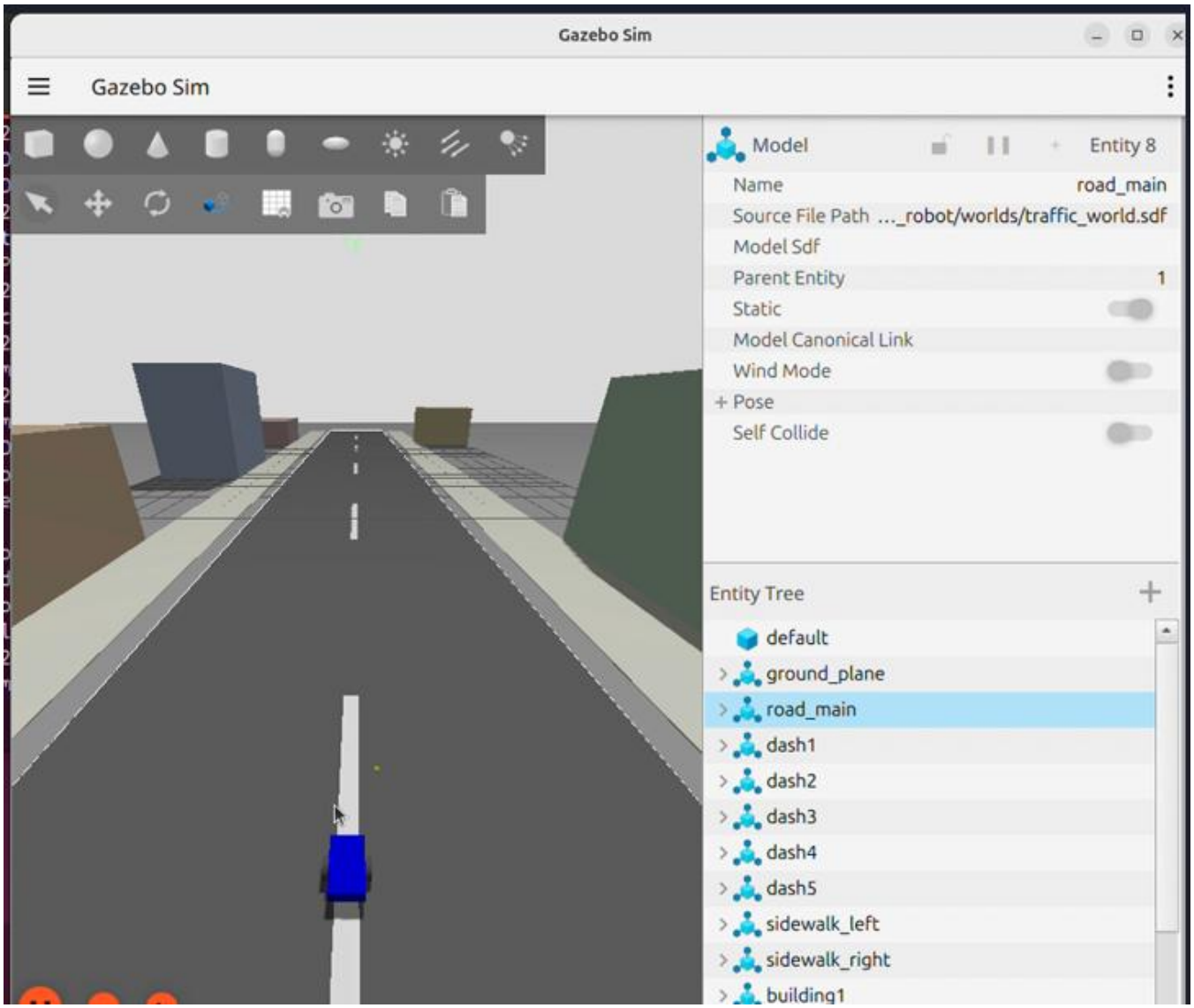
- "stop sign" → STOP
- "give way road sign" → GO
- "left only road sign" or "keep left sign" → LEFT
- "right only road sign" or "keep right sign" → RIGHT
- "speed limit 40 sign" → SLOW

- "traffic lights ahead sign" → WAIT

OUTPUT :

Link: [TRAFFIC-SIGNAL-DETECTION-ROBOT/traffic_robot_ws.zip at main · Navyashree04/TRAFFIC-SIGNAL-DETECTION-ROBOT]




Gazebo Sim
Gazebo Sim
Model
Entity 8
Name
road_main
Source File Path ..._robot/worlds/traffic_world.sdf
Model Sdf
Parent Entity
1
Static
Model Canonical Link
Wind Mode
Pose
Self Collide
Entity Tree
default
ground_plane
road_main
dash1
dash2
dash3
dash4
dash5
sidewalk_left
sidewalk_right
building1

video (1) (1).mp4